

TP1 Traceability Matrix

Requirements → Architecture → Detailed Design → Code → Test

Role: Design Documents and Architecture — integration coordination and shared infrastructure
Author: A.G. (agupt545)
Version: 1.00 **Date:** 2026-09-17
Verification status: 132 test cases, 132 passed, 0 failed (`testingAutomation.TP1RegressionSuite`, 2026-09-17)

1. How to read this matrix

Every row starts from one user story and follows it through to the test cases that prove it works. A row with an empty Test column is a row we cannot claim to have delivered, so there are none. The Test column names the test class and the numbered test cases inside it; those numbers are printed by the test driver when it runs, so a grader can match a line of this table to a line of console output during the screencast.

The architecture and design columns refer to the figures and sections of the companion document *TP1 Architecture and Detailed Design*.

Test class abbreviations

Key	Test class (package <code>testingAutomation</code>)	Cases	Covers
IV	<code>InputValidationTestingAutomation</code>	36	Every textual input validator
RM	<code>RoleManagementTestingAutomation</code>	16	Add and remove roles, last-Admin guard
IE	<code>InvitationExpirationTestingAutomation</code>	16	Invitation deadlines and expiry
IM	<code>InvitationManagementTestingAutomation</code>	20	Listing, revoking, and purging invitations
UM	<code>UserManagementTestingAutomation</code>	21	Listing users and deleting a user
OTP	<code>OneTimePasswordTestingAutomation</code>	23	One-time passwords, issue through redemption

2. Admin user stories

ID	User story	Architecture	Detailed design	Code	Test
A1	As the first person to use the system, I am made an Admin so the system is never left without one.	Application layer decides the first page (Fig. 1)	§2.1; provided design, unchanged	applicationMain.FoundationsMain, guiFirstAdmin.ControllerFirstAdmin.doSetupAdmin	IV 1-11 (the UserName and password the first Admin must supply)
A2	As an Admin, I invite a person by email address and choose the role their account will have.	Presentation → validation → persistence (Fig. 1)	§3.1, Fig. 5	guiAdminHome.ControllerAdminHome.performInvitation, .invalidEmailAddress; Database.generateInvitationCode	IV 12-27; IE 8-10; IM 2, 5
A3	As an Admin, I set a deadline on an invitation, after which the code can no longer be used.	Presentation policy method, tested without a GUI (§2.2)	§3.1; Fig. 4	ControllerAdminHome.parseInvitationDeadline, .validateInvitationDeadline, MAX_INVITATION_DAYS; Database.isInvitationExpired, .removeExpiredInvitations	IE 1-16; IM 17-19
A4	As an Admin, I list every user with their name, email address, and roles, so I can see the state of the system.	New package guiListUsers (Fig. 1)	§3.1, Fig. 2	guiListUsers.ViewListUsers, .ControllerListUsers (fullName, rolesAsText, blankIfNull); Database.getAllUsers	UM 1-9
A5	As an Admin, I delete a user, and I am asked to confirm before anything irreversible happens.	New package guiDeleteUser; policy in the controller, invariant in the database (§2.2)	§3.1, Fig. 4	guiDeleteUser.ControllerDeleteUser.checkDeletePolicy, .performDelete; Database.deleteUser	UM 10-21
A6	As an Admin, I set a one-time password for a user who cannot get in, with a deadline after which it stops working.	New package guiOneTimePassword; reuses the deadline rules (§2.2)	§3.1, Fig. 3	guiOneTimePassword.ControllerOneTimePassword.generateOneTimePassword, .performGenerate; Database.setOneTimePassword, .getOneTimePasswordDeadline, .hasActiveOneTimePassword	OTP 1-7, 14-17, 22-23
A7	As an Admin, I add and remove roles for any user in the system.	guiAddRemoveRoles (Fig. 1)	§3.1, Fig. 2	guiAddRemoveRoles.ControllerAddRemoveRoles.performAddRole, .performRemoveRole; Database.updateUserRole, .userHasRole	RM 1-5, 8-16
A8	As a stakeholder, I require that the system always keeps at least one Admin, whatever an Admin tries to do.	Defense in depth: page and database both enforce it (§2.2)	§3.1	Database.updateUserRole (Admin branch), Database.deleteUser, ControllerDeleteUser.checkDeletePolicy	RM 6-7; UM 16-19
A9	As an Admin, I review outstanding invitations and revoke one that was sent by mistake or is no longer wanted.	New package guiManageInvitations (Fig. 1)	§3.1, Fig. 4	guiManageInvitations.ControllerManageInvitations.performRevoke, .performPurge; Database.getInvitationList, .removeInvitation; entityClasses.Invitation	IM 1-20
A10	As an Admin, I am told plainly when something I asked for was refused, and why.	Presentation layer owns all messages (Fig. 1)	§3.1	Alerts in ViewAdminHome, ViewDeleteUser, ViewOneTimePassword, ViewManageInvitations; messages returned by the policy methods	UM 10-12, 19 (each returns a message, not just false); IV 27

3. New User user stories

ID	User story	Architecture	Detailed design	Code	Test
N1	As an invited person, I use my invitation code to set up an account with the role I was invited for.	Presentation → persistence (Fig. 1)	\$3.1	guiNewAccount.ViewNewAccount.displayNewAccount, .ControllerNewAccount.doCreateUser; Database.getRoleGivenAnInvitationCode, .register	IE 10; IM 5, 9
N2	As a stakeholder, I require that an expired, revoked, or unknown invitation code cannot be used to create an account.	Checked in the page and again in the database (§2.2)	\$3.1	ViewNewAccount.displayNewAccount (purge, then re-check); Database.isInvitationExpired	IE 11-15; IM 9-10
N3	As a new user, I choose a UserName, and I am told exactly what is wrong when it is not acceptable.	Validation layer, shared by three pages (Fig. 1)	\$4	userNameRecognizer.UserNameRecognizer.checkForValidUserName; called from ControllerNewAccount, ControllerFirstAdmin, ControllerUserUpdate.checkUserName	IV 1-4, 35
N4	As a new user, I choose a password, and the interface shows me which requirements I have met as I type.	Validation layer, passwordPopUpWindow (Fig. 1)	\$4.1	passwordPopUpWindow.Model.evaluatePassword, .updatePassword, View, PasswordPopupWindow.show; enforced in ControllerNewAccount.doCreateUser and ControllerFirstAdmin.doSetupAdmin	IV 5-11; OTP 2
N5	As a user, I log in with my UserName and password and reach the home page for my role.	Presentation → persistence; dispatch by role count (Fig. 1)	\$3.1	guiUserLogin.ControllerUserLogin.doLogin; Database.getUserAccountDetails, .loginAdmin, .loginContributor, .loginViewer, .loginCurator	RM 12, 16; OTP 10, 12
N6	As a user given a one-time password, I can use it once, and I must set a password of my own before I go any further.	Login page consumes what the Admin page wrote (Fig. 3)	\$3.1, \$3.2	ControllerUserLogin.doLogin, .forcePasswordReset; Database.loginWithOneTimePassword, .updatePassword, .clearOneTimePassword	OTP 8-13, 18-21
N7	As a user, I update my account details: UserName, password, names, and email address.	Presentation → validation → persistence (Fig. 1)	\$3.1, \$4.2	guiUserUpdate.ViewUserUpdate; ControllerUserUpdate.checkName, .checkUserName, .checkEmailAddress; Database.updateUserName, .updatePassword, .updateFirstName and the other setters	IV 28-36
N8	As a user who plays more than one role, I choose which role to play after logging in.	guiMultipleRoleDispatch (Fig. 1)	\$3.1	ControllerUserLogin.doLogin (role count branch); guiMultipleRoleDispatch.ViewMultipleRoleDispatch; Database.getNumberOfRoles	RM 14
N9	As a stakeholder, I require that no input field can be used to crash the application, including by pasting a very long string into it.	Size check first, in every validator (§2.2)	\$4.1, \$4.2	UserNameRecognizer, EmailAddressRecognizer.MAX_LENGTH, Model.MAX_PASSWORD_LENGTH, ControllerUserUpdate.MAX_NAME_LENGTH, ControllerUserLogin.MAX_PASSWORD_INPUT, ControllerAdminHome.parseInvitationDeadline	IV 4, 11, 25-26, 32-34; RM 10; UM 20; IM 14; OTP 20
N10	As a user, I am never shown my stored password on screen, and a failed login never tells an attacker which half was wrong.	Presentation layer decides what is displayed (Fig. 1)	\$3.1	ViewUserUpdate (MASKED_PASSWORD); ControllerUserLogin.doLogin (one message for both failures); Database.getAllUsers (no password copied out)	UM 3

4. Coverage summary

Item	Count	Note
User stories traced	20 (10 Admin, 10 New User)	Every row reaches code and at least one test case
User stories with no test coverage	0	—
Automated test cases	132	All passing as of 2026-09-17

Oversized-input test cases	11	One for each field that accepts typed text
Pages verified to build and render	13	Every page in the application constructs without error

Maintaining this matrix. A feature owner who adds or changes a user story adds or edits its row in the same merge as the code, and names the test cases that verify it. A row whose Test column would be empty is not ready to merge. This matrix is reviewed at each regression checkpoint.